

Constraint Satisfaction Problems

- ▶ CSP Consists of three component V, D, C
- ▶ V is set of variables $\{v_1, v_2, v_3, \dots, v_n\}$
- ▶ D is the set of domains $\{D_1, D_2, \dots, D_n\}$ one for each variable.
- ▶ C is set of Constraints that specify allowable combination of values.

$C_i = (\text{Scope}, \text{rel})$

Where Scope is set of variables that participate in constraint

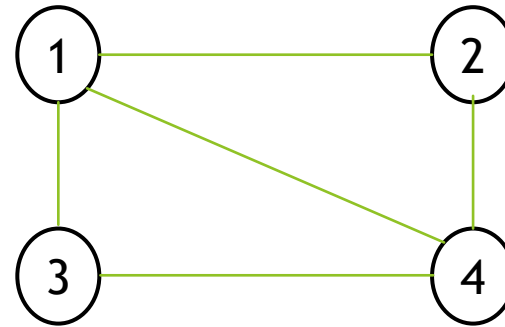
And Rel is relation that define the values that variable can take.

Example

- ▶ $V=\{v1,v2\}$
- ▶ $D=\{d1,d2\}$ where $d1=(1,2)$ and $d2=(2,4)$
- ▶ $C=(v1,v2), (d1,d2)$ ($v1$ is not equal to $v2$)
- ▶ Answer:
- ▶ $C1=(v1,v2) \quad (1,2), (1,4), (2,4)$

How Constraint Satisfaction Algorithm Works

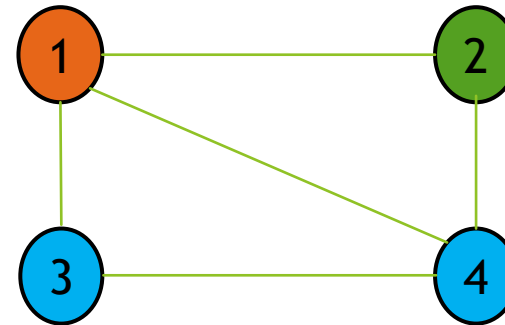
- ▶ $V=\{1,2,3,4\}$
- ▶ $D=\{\text{Red, Green, Blue}\}$
- ▶ $C= \{1 \text{ is not equal to } 2\}$
 - $\{1 \text{ is not equal to } 3\}$
 - $\{1 \text{ is not equal to } 4\}$
 - $\{2 \text{ is not equal to } 4\}$
 - $\{3 \text{ is not equal to } 4\}$



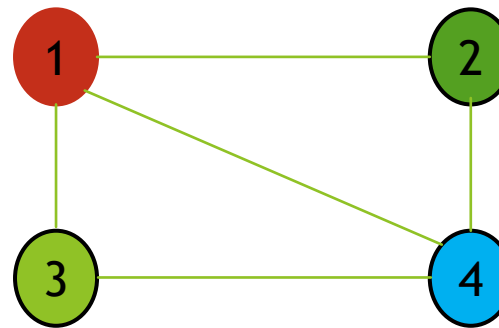
How Constraint Satisfaction Algorithm Works

		1	2	3	4
Step1	Initial	RGB	RGB	RGB	RGB
Step2	1=R	R	GB	GB	GB
Step3	2=G	R	G	GB	B
Step3 Backtracking	3=B	R	G	B	B
Step4	3=G	R	G	G	B

► Wrong



► Right



Binary Search tree

- ▶ **Tree**

- ▶ A tree is a kind of data structure that is used to represent the data in hierarchical form. It can be defined as a collection of objects or entities called as nodes that are linked together to simulate a hierarchy. Tree is a non-linear data structure as the data in a tree is not stored linearly or sequentially.

- ▶ **Binary Search tree**

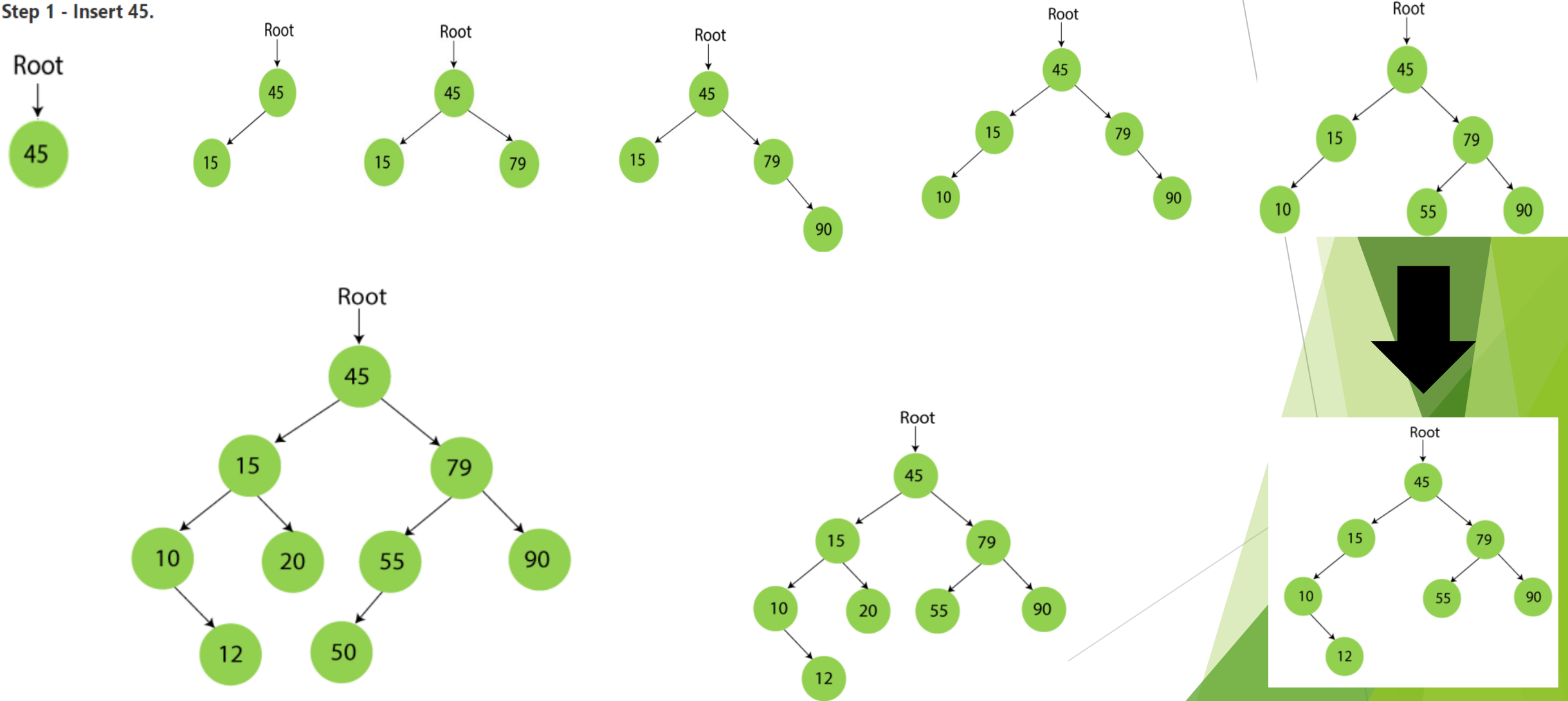
- ▶ A binary search tree follows some order to arrange the elements. In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.

Example of creating a binary search tree

- ▶ Suppose the data elements are - **45, 15, 79, 90, 10, 55, 12, 20, 50**
 - First, we have to insert **45** into the tree as the root of the tree.
 - Then, read the next element; if it is smaller than the root node, insert it as the root of the left subtree, and move to the next element.
 - Otherwise, if the element is larger than the root node, then insert it as the root of the right subtree.

data elements are - **45, 15, 79, 90, 10, 55, 12, 20, 50**

Step 1 - Insert 45.



Searching in Binary search tree

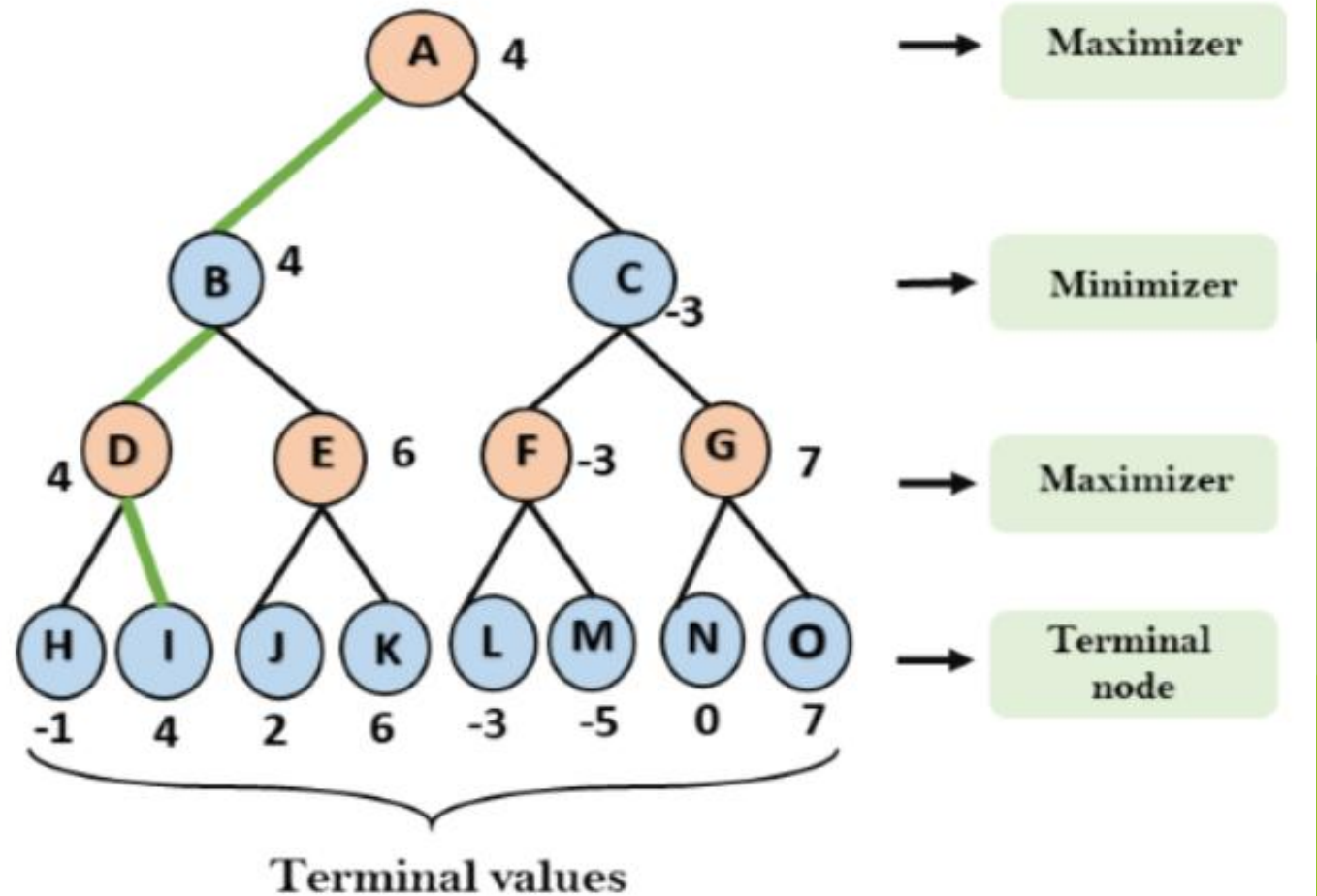
1. First, compare the element to be searched with the root element of the tree.
2. If root is matched with the target element, then return the node's location.
3. If it is not matched, then check whether the item is less than the root element, if it is smaller than the root element, then move to the left subtree.
4. If it is larger than the root element, then move to the right subtree.
5. Repeat the above procedure recursively until the match is found.
6. If the element is not found or not present in the tree, then return NULL.

Mini-Max Algorithm

- Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory. It provides an optimal move for the player assuming that opponent is also playing optimally.
- Mini-Max algorithm uses recursion to search through the game-tree.
- Min-Max algorithm is mostly used for game playing in AI. Such as Chess, Checkers, tic-tac-toe, go, and various tow-players game. This Algorithm computes the minimax decision for the current state.
- In this algorithm two players play the game, one is called MAX and other is called MIN.
- Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.
- Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.
- The minimax algorithm performs a depth-first search algorithm for the exploration of the complete game tree.
- The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

Mini-Max Algorithm

- ▶ Backtracking algorithm used
- ▶ Best move strategy used
- ▶ Max will try to maximize its utility(Best move)
- ▶ Min will try to minimize Utility(Worst move)

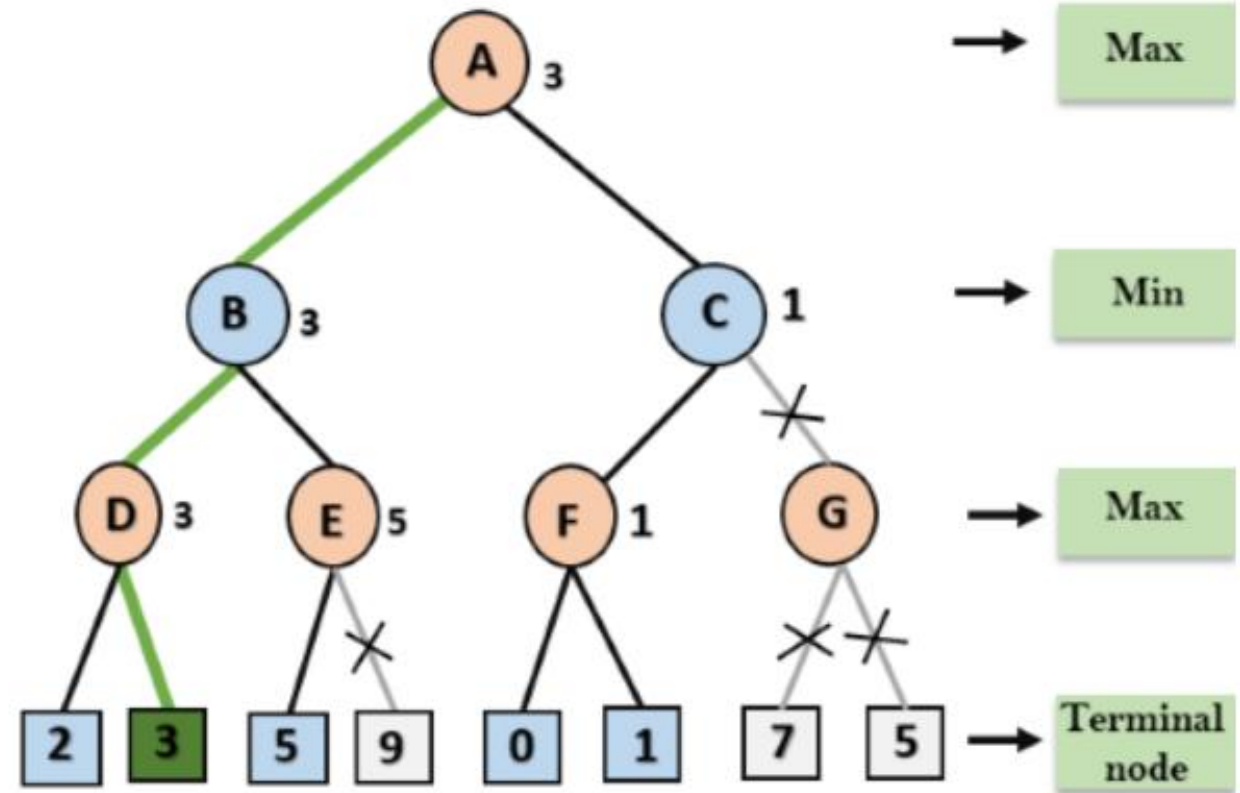


Alpha-Beta Pruning

- Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimization technique for the minimax algorithm.
- As we have seen in the minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree. Since we cannot eliminate the exponent, but we can cut it to half. Hence there is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called **pruning**. This involves two threshold parameter Alpha and beta for future expansion, so it is called **alpha-beta pruning**. It is also called as **Alpha-Beta Algorithm**.
- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prune the tree leaves but also entire sub-tree.
- The two-parameter can be defined as:
 - **Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is $-\infty$.
 - **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is $+\infty$.
- The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not really affecting the final decision but making algorithm slow. Hence by pruning these nodes, it makes the algorithm fast.

Alpha-Beta Pruning

- ▶ Key points about alpha-beta pruning:
 - The Max player will only update the value of alpha.
 - The Min player will only update the value of beta.
 - While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
 - We will only pass the alpha, beta values to the child nodes.



Propositional logic in Artificial intelligence

- ▶ Propositional logic (PL) is the simplest form of logic where all the statements are made by propositions. A proposition is a declarative statement which is either true or false. It is a technique of knowledge representation in logical and mathematical form.

- ▶ **Example:**

1. a) It is Sunday.
2. b) The Sun rises from West (False proposition)
3. c) $3+3=7$ (False proposition)
4. d) 5 is a prime number.

- ▶ **Following are some basic facts about propositional logic:**

- Propositional logic is also called Boolean logic as it works on 0 and 1.
- In propositional logic, we use symbolic variables to represent the logic, and we can use any symbol for a representing a proposition, such A, B, C, P, Q, R, etc.
- Propositions can be either true or false, but it cannot be both.
- Propositional logic consists of an object, relations or function, and **logical connectives**.

- These connectives are also called logical operators.
- The propositions and connectives are the basic elements of the propositional logic.
- Connectives can be said as a logical operator which connects two sentences.
- A proposition formula which is always true is called **tautology**, and it is also called a valid sentence.
- A proposition formula which is always false is called **Contradiction**.
- Statements which are questions, commands, or opinions are not propositions such as "**Where is Rohini**", "**How are you**", "**What is your name**", are not propositions.

Syntax of propositional logic:

- ▶ The syntax of propositional logic defines the allowable sentences for the knowledge representation. There are two types of Propositions:
 1. **Atomic Propositions**
 2. **Compound propositions**
- **Atomic Proposition:** Atomic propositions are the simple propositions. It consists of a single proposition symbol. These are the sentences which must be either true or false.
 - ▶ **Example:**
 1. a) $2+2$ is 4 , it is an atomic proposition as it is a **true** fact.
 2. b) "The Sun is cold" is also a proposition as it is a **false** fact.
- **Compound proposition:** Compound propositions are constructed by combining simpler or atomic propositions, using parenthesis and logical connectives.
 - ▶ **Example:**
 1. a) "It is raining today, and street is wet."
 2. b) "Ankit is a doctor, and his clinic is in Mumbai."

Logical Connectives

- ▶ Logical connectives are used to connect two simpler propositions or representing a sentence logically. We can create compound propositions with the help of logical connectives. There are mainly five connectives, which are given as follows:

- ▶ **Negation:** A sentence such as $\neg P$ is called negation of P. A literal can be either Positive literal or negative literal.

- ▶ **Conjunction:** A sentence which has $\wedge$ connective such as, $P \wedge Q$ is called a conjunction.

Example: Rohan is intelligent and hardworking. It can be written as,

P= Rohan is intelligent,

Q= Rohan is hardworking. $\rightarrow P \wedge Q$.

- ▶ **Disjunction:** A sentence which has $\vee$ connective, such as $P \vee Q$. is called disjunction, where P and Q are the propositions.

Example: "Ritika is a doctor or Engineer",

Here P= Ritika is Doctor. Q= Ritika is Doctor, so we can write it as $P \vee Q$.

- ▶ **Implication:** A sentence such as $P \rightarrow Q$, is called an implication. Implications are also known as if-then rules. It can be represented as

If it is raining, then the street is wet.

Let P= It is raining, and Q= Street is wet, so it is represented as $P \rightarrow Q$

- ▶ **Biconditional:** A sentence such as $P \Leftrightarrow Q$ is a Biconditional sentence, example If I am breathing, then I am alive

P= I am breathing, Q= I am alive, it can be represented as $P \Leftrightarrow Q$.

Truth Table:

For Negation:

P	$\neg P$
True	False
False	True

For Conjunction:

P	Q	$P \wedge Q$
True	True	True
True	False	False
False	True	False
False	False	False

For disjunction:

P	Q	$P \vee Q$
True	True	True
False	True	True
True	False	True
False	False	False

Truth Table:

For Implication:

P	Q	$P \rightarrow Q$
True	True	True
True	False	False
False	True	True
False	False	True

For Biconditional:

P	Q	$P \leftrightarrow Q$
True	True	True
True	False	False
False	True	False
False	False	True

First-Order logic:

- First-order logic is another way of knowledge representation in artificial intelligence. It is an extension to propositional logic.
- FOL is sufficiently expressive to represent the natural language statements in a concise way.
- First-order logic is also known as **Predicate logic** or **First-order predicate logic**. First-order logic is a powerful language that develops information about the objects in a more easy way and can also express the relationship between those objects.
- First-order logic (like natural language) does not only assume that the world contains facts like propositional logic but also assumes the following things in the world:
 - **Objects:** A, B, people, numbers, colors, wars, theories, squares, pits, wumpus,
 - **Relations:** It can be **unary relation such as:** red, round, is adjacent, **or n-any relation such as:** the sister of, brother of, has color, comes between
 - **Function:** Father of, best friend, third inning of, end of,
- As a natural language, first-order logic also has two main parts:
 - **Syntax**
 - **Semantics**

Basic Elements of First-order logic:

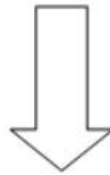
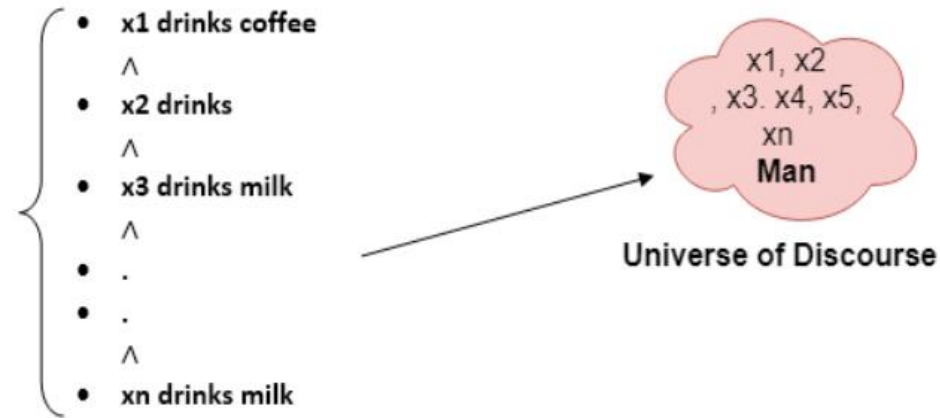
Constant	1, 2, A, John, Mumbai, cat,....
Variables	x, y, z, a, b,....
Predicates	Brother, Father, >,....
Function	sqrt, LeftLegOf,
Connectives	$\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\Leftrightarrow$
Equality	$=$
Quantifier	$\forall$, $\exists$

Quantifiers in First-order logic:

- A quantifier is a language element which generates quantification, and quantification specifies the quantity of specimen in the universe of discourse.
- These are the symbols that permit to determine or identify the range and scope of the variable in the logical expression. There are two types of quantifier:
 - **Universal Quantifier, (for all, everyone, everything)**
 - **Existential quantifier, (for some, at least one).**

Universal Quantifier:

- ▶ Universal quantifier is a symbol of logical representation, which specifies that the statement within its range is true for everything or every instance of a particular thing.
- ▶ The Universal quantifier is represented by a symbol $\forall$, which resembles an inverted A.
- ▶ If x is a variable, then $\forall x$ is read as:
 - **For all x**
 - **For each x**
 - **For every x .**
- ▶ **Example:**
- ▶ **All man drink coffee.**



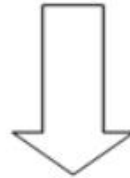
So in shorthand notation, we can write it as :

$$\forall x \text{ man}(x) \rightarrow \text{drink}(x, \text{coffee}).$$

Existential Quantifier

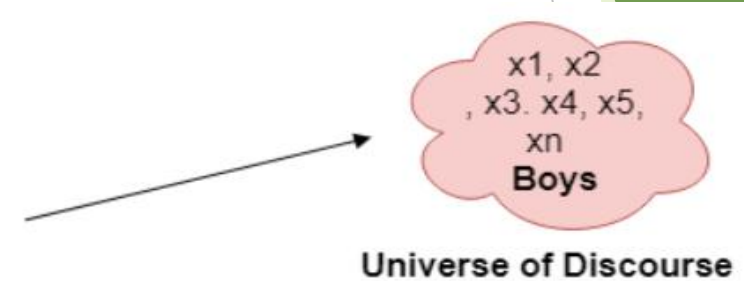
- ▶ Existential quantifiers are the type of quantifiers, which express that the statement within its scope is true for at least one instance of something.
- ▶ It is denoted by the logical operator $\exists$, which resembles as inverted E. When it is used with a predicate variable then it is called as an existential quantifier.
- ▶ If x is a variable, then existential quantifier will be $\exists x$ or $\exists(x)$. And it will be read as:
 - **There exists a 'x.'**
 - **For some 'x.'**
 - **For at least one 'x.'**
- ▶ **Example:**
- ▶ **Some boys are intelligent.**

- **x1 is intelligent**
 $\vee$
- **x2 is intelligent** $\vee$
- **x3 is intelligent**
 $\vee$
- **.**
- **.**
 $\vee$
- **xn is intelligent**



So in short-hand notation, we can write it as:

$\exists x: \text{boys}(x) \wedge \text{intelligent}(x)$



Some Examples of First-order logic using quantifier:

► 1. All birds fly.

In this question the predicate is "fly(bird)."

And since there are all birds who fly so it will be represented as follows.

$$\forall x \text{ bird}(x) \rightarrow \text{fly}(x).$$

► 2. Every man respects his parent.

In this question, the predicate is "respect(x, y)," where x=man, and y= parent.

Since there is every man so will use $\forall$, and it will be represented as follows:

$$\forall x \text{ man}(x) \rightarrow \text{respects}(x, \text{parent}).$$

► 3. Some boys play cricket.

In this question, the predicate is "play(x, y)," where x= boys, and y= game. Since there are some boys so we will use $\exists$, and it will be represented as:

$$\exists x \text{ boys}(x) \rightarrow \text{play}(x, \text{cricket}).$$

► 4. Not all students like both Mathematics and Science.

In this question, the predicate is "like(x, y)," where x= student, and y= subject.

Since there are not all students, so we will use $\forall$ with negation, so following representation for this:

$$\neg \forall (x) [\text{student}(x) \rightarrow \text{like}(x, \text{Mathematics}) \wedge \text{like}(x, \text{Science})].$$

► 5. Only one student failed in Mathematics.

In this question, the predicate is "failed(x, y)," where x= student, and y= subject.

Since there is only one student who failed in Mathematics, so we will use following representation for this:

$$\exists (x) [\text{student}(x) \rightarrow \text{failed}(x, \text{Mathematics}) \wedge \forall (y) [\neg(x=y) \wedge \text{student}(y) \rightarrow \neg \text{failed}(y, \text{Mathematics})]].$$

Forward Chaining and backward chaining

► Inference engine:

- The inference engine is the component of the intelligent system in artificial intelligence, which applies logical rules to the knowledge base to infer new information from known facts. The first inference engine was part of the expert system. Inference engine commonly proceeds in two modes, which are:
 1. **Forward chaining**
 2. **Backward chaining**

Forward Chaining

- It is a down-up approach, as it moves from bottom to top.
- It is a process of making a conclusion based on known facts or data, by starting from the initial state and reaches the goal state.
- Forward-chaining approach is also called as data-driven as we reach to the goal using available data.
- Forward -chaining approach is commonly used in the expert system, such as CLIPS, business, and production rule systems.
- ▶ **Example:**
"As per the law, it is a crime for an American to sell weapons to hostile nations. Country A, an enemy of America, has some missiles, and all the missiles were sold to it by Robert, who is an American citizen."
- ▶ Prove that "**Robert is criminal.**"

Facts Conversion into FOL:

- It is a crime for an American to sell weapons to hostile nations. (Let's say p, q, and r are variables)

American (p) $\wedge$ weapon(q) $\wedge$ sells (p, q, r) $\wedge$ hostile(r) $\rightarrow$ Criminal(p) ... (1)

- Country A has some missiles. **?p Owns(A, p) $\wedge$ Missile(p)**. It can be written in two definite clauses by using Existential Instantiation, introducing new Constant T1.

Owns(A, T1) (2)

Missile(T1) (3)

- All of the missiles were sold to country A by Robert.

?p Missiles(p) $\wedge$ Owns (A, p) $\rightarrow$ Sells (Robert, p, A) (4)

- Missiles are weapons.

Missile(p) $\rightarrow$ Weapons (p) (5)

- Enemy of America is known as hostile.

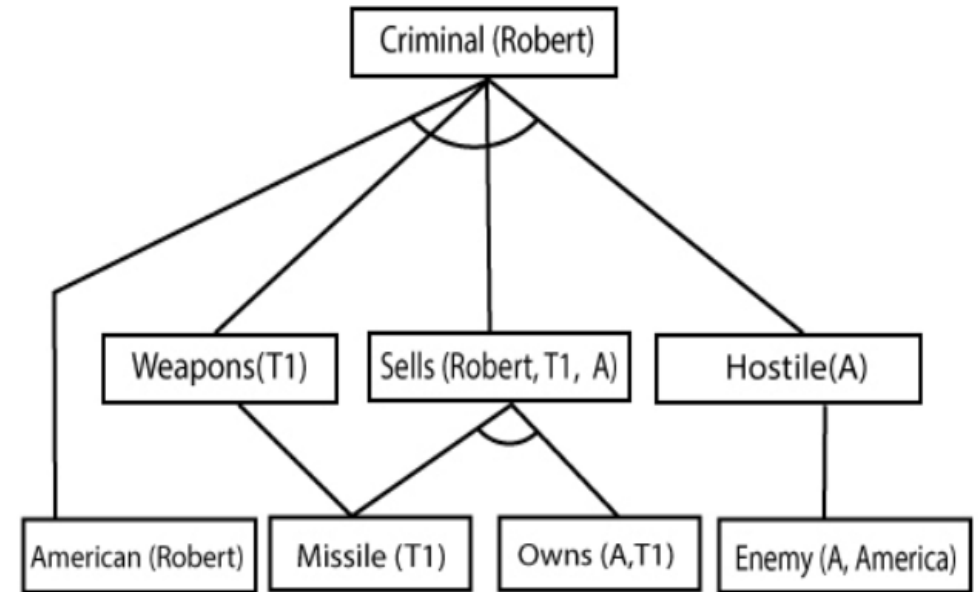
Enemy(p, America) $\rightarrow$ Hostile(p) (6)

- Country A is an enemy of America.

Enemy (A, America) (7)

- Robert is American

American(Robert). (8)



Hence it is proved that Robert is Criminal using forward chaining approach.

Backward chaining

- It is known as a top-down approach.
- Backward-chaining is based on modus ponens inference rule.
- In backward chaining, the goal is broken into sub-goal or sub-goals to prove the facts true.
- It is called a goal-driven approach, as a list of goals decides which rules are selected and used.
- Backward -chaining algorithm is used in game theory, automated theorem proving tools, inference engines, proof assistants, and various AI applications.
- The backward-chaining method mostly used a **depth-first search** strategy for proof.

Example:

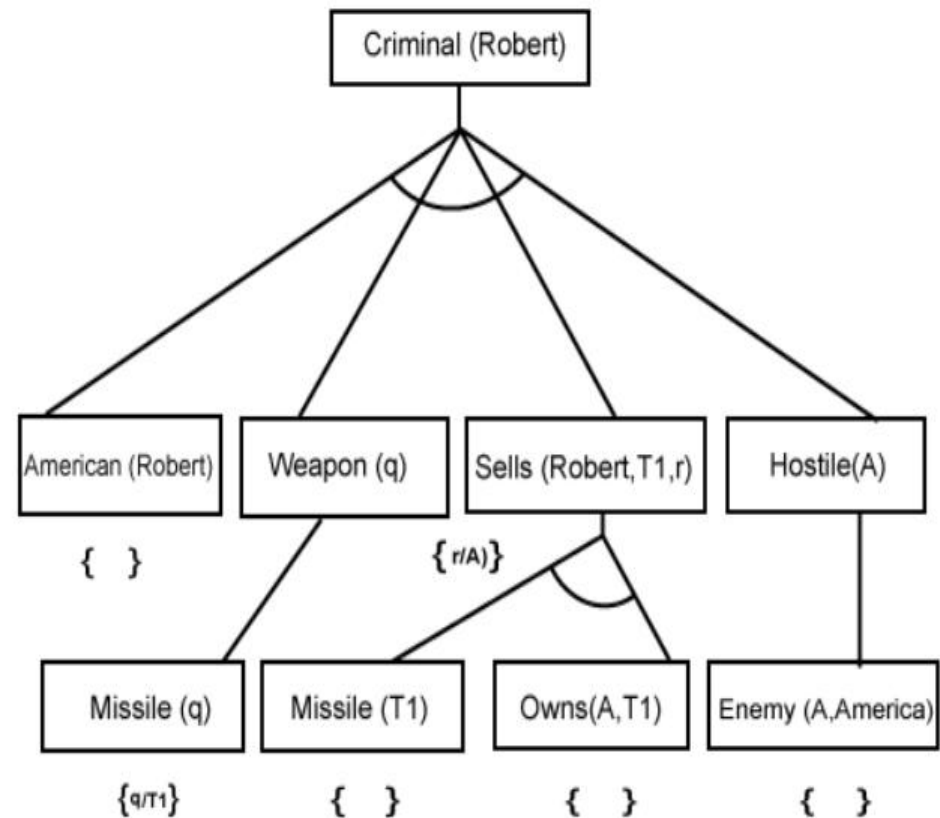
"As per the law, it is a crime for an American to sell weapons to hostile nations. Country A, an enemy of America, has some missiles, and all the missiles were sold to it by Robert, who is an American citizen."

Prove that "Robert is criminal."

Example:

In backward-chaining, we will use the same above example, and will rewrite all the rules.

- $\text{American}(p) \wedge \text{weapon}(q) \wedge \text{sells}(p, q, r) \wedge \text{hostile}(r) \rightarrow \text{Criminal}(p) \dots(1)$
- $\text{Owns}(A, T1) \dots\dots\dots(2)$
- $\text{Missile}(T1)$
- $?p \text{ Missiles}(p) \wedge \text{Owns}(A, p) \rightarrow \text{Sells}(\text{Robert}, p, A) \dots\dots\dots(4)$
- $\text{Missile}(p) \rightarrow \text{Weapons}(p) \dots\dots\dots(5)$
- $\text{Enemy}(p, \text{America}) \rightarrow \text{Hostile}(p) \dots\dots\dots(6)$
- $\text{Enemy}(A, \text{America}) \dots\dots\dots(7)$
- $\text{American}(\text{Robert}). \dots\dots\dots(8)$



Probabilistic reasoning:

- ▶ Probabilistic reasoning is a way of knowledge representation where we apply the concept of probability to indicate the uncertainty in knowledge. In probabilistic reasoning, we combine probability theory with logic to handle the uncertainty.
- ▶ We use probability in probabilistic reasoning because it provides a way to handle the uncertainty that is the result of someone's laziness and ignorance.
- ▶ In the real world, there are lots of scenarios, where the certainty of something is not confirmed, such as "It will rain today," "behavior of someone for some situations," "A match between two teams or two players." These are probable sentences for which we can assume that it will happen but not sure about it, so here we use probabilistic reasoning.
- ▶ **Need of probabilistic reasoning in AI:**
 - When there are unpredictable outcomes.
 - When specifications or possibilities of predicates becomes too large to handle.
 - When an unknown error occurs during an experiment.
- ▶ In probabilistic reasoning, there are two ways to solve problems with uncertain knowledge:
 - **Bayes' rule**
 - **Bayesian Statistics**

Probability

- ▶ Probability can be defined as a chance that an uncertain event will occur. It is the numerical measure of the likelihood that an event will occur. The value of probability always remains between 0 and 1 that represent ideal uncertainties.
- 1. $0 \leq P(A) \leq 1$, where $P(A)$ is the probability of an event A.
- 2. $P(A) = 0$, indicates total uncertainty in an event A.
- 3. $P(A) = 1$, indicates total certainty in an event A.
- ▶ We can find the probability of an uncertain event by using the below formula.

$$\text{Probability of occurrence} = \frac{\text{Number of desired outcomes}}{\text{Total number of outcomes}}$$

- $P(\neg A)$ = probability of a not happening event.
- $P(\neg A) + P(A) = 1$.

Conditional probability:

Conditional probability is a probability of occurring an event when another event has already happened.

Let's suppose, we want to calculate the event A when event B has already occurred, "the probability of A under the conditions of B", it can be written as:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

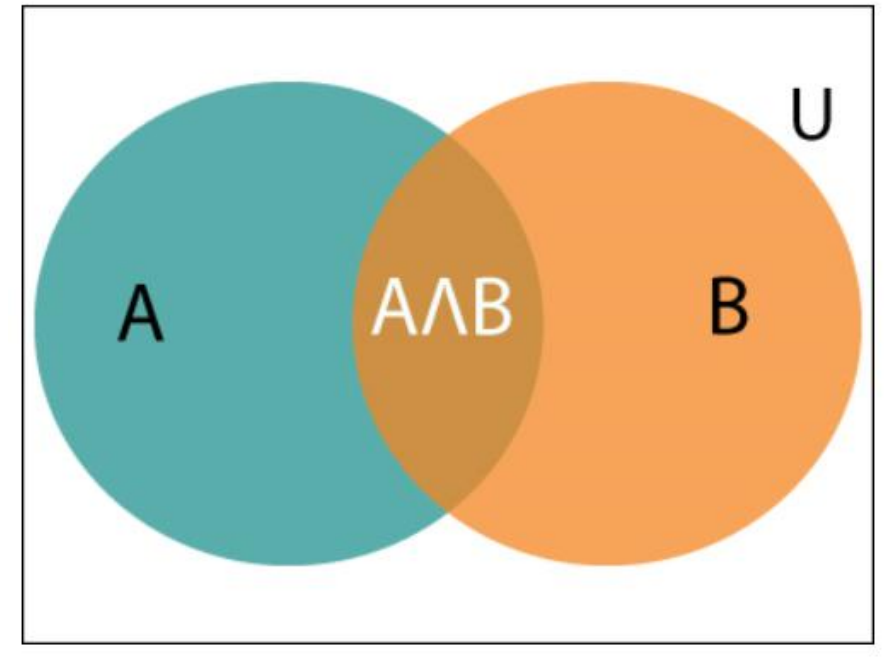
Where $P(A \cap B)$ = Joint probability of a and B

$P(B)$ = Marginal probability of B.

If the probability of A is given and we need to find the probability of B, then it will be given as:

$$P(B|A) = \frac{P(A \cap B)}{P(A)}$$

It can be explained by using the below Venn diagram, where B is occurred event, so sample space will be reduced to set B, and now we can only calculate event A when event B is already occurred by dividing the probability of $P(A \cap B)$ by $P(B)$.



Example:

In a class, there are 70% of the students who like English and 40% of the students who likes English and mathematics, and then what is the percent of students those who like English also like mathematics?

Solution:

Let, A is an event that a student likes Mathematics

B is an event that a student likes English.

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{0.4}{0.7} = 57\%$$

Hence, 57% are the students who like English also like Mathematics.

Bayes' theorem

- ▶ Bayes' theorem is also known as **Bayes' rule**, **Bayes' law**, or **Bayesian reasoning**, which determines the probability of an event with uncertain knowledge.
- ▶ In probability theory, it relates the conditional probability and marginal probabilities of two random events.
- ▶ Bayes' theorem was named after the British mathematician **Thomas Bayes**. The **Bayesian inference** is an application of Bayes' theorem, which is fundamental to Bayesian statistics.

Bayes' theorem

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

$$P(\text{cause}|\text{effect}) = \frac{P(\text{effect}|\text{cause}) P(\text{cause})}{P(\text{effect})}$$

- ▶ $P(A|B)$ is known as **posterior**, which we need to calculate, and it will be read as Probability of hypothesis A when we have occurred an evidence B.
- ▶ $P(B|A)$ is called the **likelihood**, in which we consider that hypothesis is true, then we calculate the probability of evidence.
- ▶ $P(A)$ is called the **prior probability**, probability of hypothesis before considering the evidence
- ▶ $P(B)$ is called **marginal probability**, pure probability of an evidence.

Bayes' theorem Example

Question: From a standard deck of playing cards, a single card is drawn. The probability that the card is king is $4/52$, then calculate posterior probability $P(\text{King}|\text{Face})$, which means the drawn face card is a king card.

Solution:

$$P(\text{king}|\text{face}) = \frac{P(\text{Face}|\text{king}) \cdot P(\text{King})}{P(\text{Face})} \dots\dots(i)$$

$P(\text{king})$: probability that the card is King = $4/52 = 1/13$

$P(\text{face})$: probability that a card is a face card = $3/13$

$P(\text{Face}|\text{King})$: probability of face card when we assume it is a king = 1

Putting all values in equation (i) we will get:

$$P(\text{king}|\text{face}) = \frac{1 * (\frac{1}{13})}{(\frac{3}{13})} = 1/3, \text{ it is a probability that a face card is a king card.}$$

The background features abstract, overlapping geometric shapes in various shades of green, primarily on the right side, creating a modern, layered effect. The rest of the background is a solid light gray.

Thank You